

模块一：统计 32 位二进制的 0 和 1 的个数

思路：随机一个 32 位的二进制数，然后用 always 语句，对这个二进制数做一个 for 循环，统计出现的 1 的个数，然后 32-1 的个数得到 0 的个数。

```
module test(
);
reg [31:0] num;
wire [5:0] count1;
wire [5:0] count0;
count32 c(num, count1, count0);
always #10
begin
assign num = $random %33'b1_0000_0000_0000_0000_0000_0000_0000;
end
endmodule
```

```
module count32(
input [31:0] num,
output [5:0] count0,
output [5:0] count1
);
reg [5:0] counttmp1;
integer i;

always @(*) begin
counttmp1 = 0;
for (i = 0; i < 32; i = i + 1) begin
counttmp1 = counttmp1 + num[i];
end
end
assign count1 = counttmp1;
assign count0 = 32 - count1;
endmodule
```

图 1（右）模块实现代码

图 2（左）仿真代码

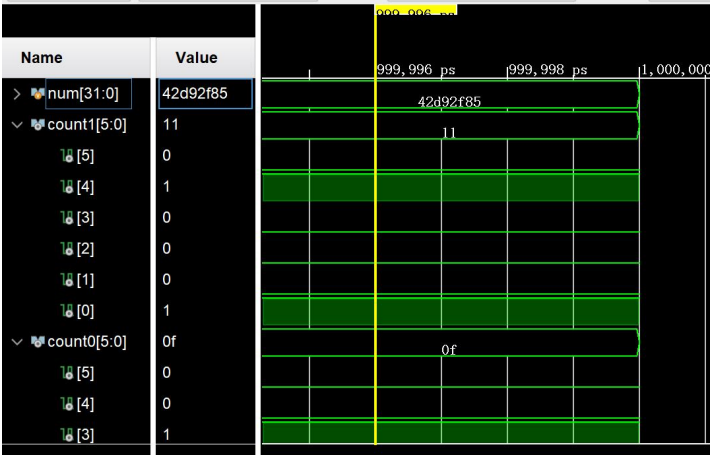


图 3 仿真结果

模块二：十分频器

思路：传入时钟和置位信号，用 always 检测时钟信号和置位信号（这里均检验上升沿），如果置位信号为 1 则重置，否则则每次正沿就计数，因为是用阻塞赋值，所以为 4 时输出时钟取反。

```

module divider10(
input clk_in,
input reset,
output reg [4:0]count,
output reg clk_out
);
always @(posedge clk_in or posedge reset) begin
    if (reset) begin
        count <= 0;
        clk_out <= 0;
    end
    else
    begin
        count = count + 1;
        if(count == 5)begin
            count <= 0;
            clk_out <= ~clk_out;
        end
    end
end
endmodule

```

```

module test(
);
reg clk_in;
reg reset;
wire [4:0] count;
wire clk_out;

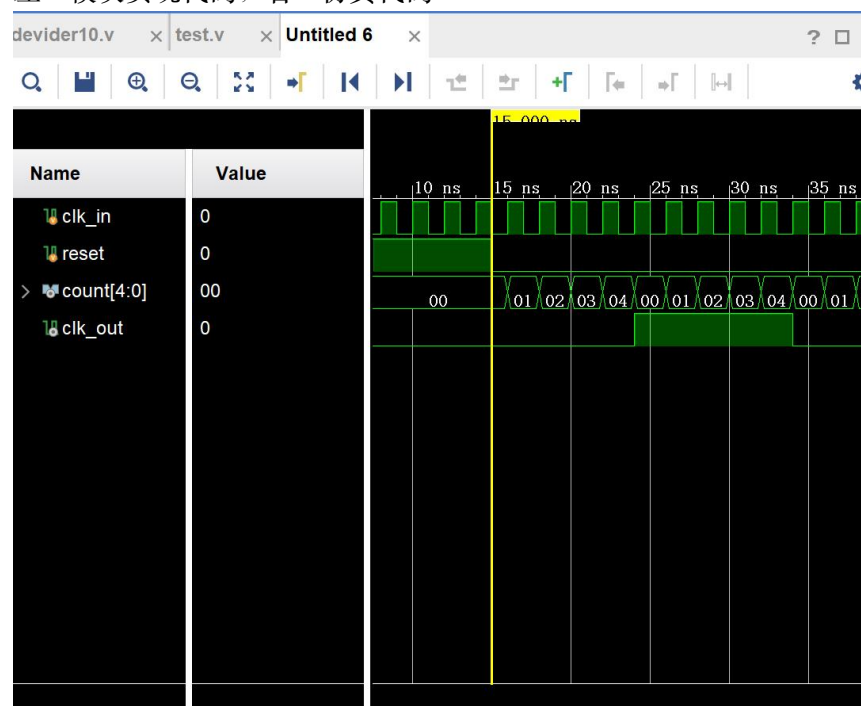
always begin
    clk_in = 1;
    #1;
    clk_in = 0;
    #1;
end

divider10 de(clk_in, reset, count, clk_out);

initial begin
    reset =1;
    #15;
    reset = 0;
    #1000;
    $stop;
end

```

左：模块实现代码；右：仿真代码



仿真结果图

心得体会

本次实验，首次使用了时钟，对时钟的初始化以及对 always 语句的使用更加熟练。在实验中还阻塞赋值和非阻塞赋值有了更加清晰的认识。课上也学习了许多预处理的语句，在第二个模块中也用到了对时间的设定。